Logical Entailment Reasoning Agent

AI Assignment - P26AI0009

Introduction

This is a system that reads a knowledge base written in plain English, reads a query, and decides whether the knowledge base entails the query. My system follows the pipeline of natural language, then an internal representation, then classical reasoning, then True / False. The conversion from English into the internal representation is done by a parser , and the final True / False decision always comes from a forward-chaining engine. The system was developed and evaluated on the three public datasets, namely RuleTaker, CounterBench, and FOLIO. Everything below is implemented in Python in the attached notebook reasoning_agent.ipynb, which prints the final accuracy and the total inference time and writes submission.csv when runs.

Knowledge Representation

All sentences are turned into one single internal format. A literal is a tuple (predicate,arguments, negated). A fact is a literal with no variables, and a rule is a pair (antecedents, consequents) whose arguments may be variables written like ?x. Variables are matched against the constants of the knowledge base, which is exactly how classic rule systems bind variables. The table shows how the parser reads the most common sentence shapes:

English Sentence	**Internal Form**
Bob is kind.	kind(bob)
The dog is not green	Not green(dog)
The bear eats the mouse	eats(bear,mouse)
If someone is young then they are smart	young(?x) -> smart(?x)
If someone is smart and kind then they are successful	smart(?x) and kind(?x) -> successful(?x)
All green things are cold	green(?x) -> cold(?x)
Ziklo causes Blaf	occurs(ziklo) -> occurs(blaf) and not occurs(ziklo) -> not occurs(blaf)

The parser is deliberately conservative where a sentence it cannot confidently match is skipped instead of guessed. That way the engine can only under-derive, never derive something the text does not support. CounterBenchmechanisms are stored as two directions of rules (A causes B becomes both A to B and not-B to not-A),which makes the causal model deterministic in both directions, exactly as the dataset intends.

Reasoning Algorithm

The engine is a classic forward chainer. It keeps the known facts, fires every rule whose antecedents are satisfied, adds the instantiated conclusions as new facts, and repeats until nothing changes. Each round derives everything that needs one more rule application, so the search is breadth-first in derivation depth, facts that need fewer steps are found first. For every derived fact the engine records the rule and the supporting facts, which is what makes the printed reasoning traces possible.

Negation. RuleTaker labels use the standard logic-programming reading (negation as failure, as in Prolog): a negative statement such as "The lion is not cold" holds when cold(lion) cannot be derived, and rules with negative conditions fire on that basis. FOLIO, in contrast, is judged with the classical open-world reading: a negative literal only holds when it is explicitly derived. Both modes are one flag in the engine.

Counterfactuals. A CounterBench question such as "Would Lumbo occur if not Ziklo?" is answered with the same forward chainer: the intervention is added as a fact not-occurs(ziklo), the rule that would re-derive the intervened variable is dropped (Pearl's truncation), and the change is propagated through the causal rules. Observations ("We observed Trune") are pinned to their observed value. The answer is "yes" when the target is derived, otherwise "no".

Final decision. The decision rule per dataset: RuleTaker - True, when the query literal is derivable, and for a negated query True when its positive part is not derivable; CounterBench, "yes" (True) when the target occurs in the counterfactual world; FOLIO - True only when the conclusion itself is derived and its negation is not, everything else counts as not entailed (which also covers FOLIO's third label Uncertain.

Experiments on public datasets

The evaluation uses 800 queries sampled from the RuleTaker test split, all 1,000 CounterBench queries, and the full FOLIO validation set (203 conclusions) - 2,003 queries in total, with a fixed random seed so every run is reproducible.

Overall Accuracy : 94.5% (1892/2003 queries)
Total Inference time : 0.49 s
Average reasoning time per Query : 0.2 ms

Dataset	Queries	Correct	Accuracy	Reasoning Time
RuleTaker	800	800	100%	0.35s
CounterBench	1000	960	96%	0.11s
FOLIO	203	132	65%	0.03s
Total	2003	1892	94.5%	0.49s

RuleTaker (100.0%). The stories use a small set of sentence templates, which the parser covers fully, and forward chaining reproduces the dataset's own negation-as-failure labelling. All 800 sampled queries are answered correctly, including depth-5 stories that need chains of five rule applications.

CounterBench (96.0%). Basic questions are answered perfectly. The remaining 40 errors sit in the conditional / joint / nested types, where a few of the generated gold answers disagree with the structural causal model that the text describes - the paper's own best method (CoIn) also stays below 100% on those types. I verified the semantics against the official algorithm description, so I treat this residual as dataset noise rather than a reasoning error.

FOLIO (65.0%). FOLIO conclusions often need disjunction, existentials or paraphrase-level matching ("included in the Top 10 list" vs "on the Top 10 list"), which a Horn-form forward chainer cannot express. The parser skips such sentences (staying sound), so FOLIO accuracy lands near the "not entailed" baseline while all template-derivable conclusions are found.

Reasoning Traces

The engine records which rule produced which fact, so every answer can be shown as a proof tree. "a <- rule: b -> a" means the fact a was derived by that rule.

a)Trace 1 - the worked example from the assignment sheet
knowledge base: Bob is young. Bob is kind.
If someone is young then they are smart.
If someone is smart and kind then they are successful.
query: Bob is successful. predicted: True
successful(bob) <- rule: young(?x) and kind(?x) -> successful(?x)
smart(bob) <- rule: young(?x) -> smart(?x)
young(bob) <- given as a fact
kind(bob) <- given as a fact

The next trace shows a five-hop RuleTaker derivation (abbreviated): being nice makes you need the lion, needing the lion while nice makes you visit the squirrel, visiting the squirrel makes the squirrel young, and a young individual that needs the lion finally makes the lion visit the cat.

b) Trace 2 - multi-hop RuleTaker proof

query: The lion visits the cat. predicted: True
visit(lion, cat) <- rule: young(?x) and need(?x, lion) -> visit(lion, cat)
young(squirrel) <- rule: visit(?x, squirrel) -> young(squirrel)
visit(squirrel, squirrel) <- rule: nice(?x) and need(?x, lion) -> visit(?x, squirrel)
nice(squirrel) <- given as a fact
need(squirrel, lion) <- rule: nice(?x) -> need(?x, lion)
need(squirrel, lion) <- rule: nice(?x) -> need(?x, lion)

c) Trace 3 - negation as failure

query: The lion is not cold. predicted: True
not cold(lion) holds because cold(lion) is NOT derivable (negation as failure).
the only cold fact in the story is cold(tiger);
no rule mentions cold for anyone else.

d) Trace 4 - CounterBench counterfactual

mechanisms: Ziklo causes Blaf, Blaf causes Trune, Trune causes Vork,
and Vork causes Lumbo.
question: Would Lumbo occur if not Ziklo instead of Ziklo? gold: no
intervention: occurs(ziklo) = false; the rule occurs(ziklo) -> occurs(blaf)
is dropped (truncation); propagation gives: not occurs(ziklo) -> not occurs(blaf) -> not
occurs(trune) -> not occurs(vork) -> not occurs(lumbo)
answer: Lumbo does not occur -> prediction False (correct)

Limits

The system is deliberately simple: a template parser plus one forward-chaining engine with a negation flag. That simplicity is also its main limitation. FOLIO would need disjunction and existential handling (skolemisation exists in the parser but the conclusions rarely match), and richer paraphrase normalisation of predicate names; both are natural next steps. On the data whose labels follow rule-based semantics (RuleTaker, CounterBench), the system is at or near the ceiling, and it stays fast: the whole 2,003-query evaluation takes about half a second, i.e. roughly 0.2 ms of reasoning per query, because rules are matched against a small, indexed fact set.